

Algoritmos

Sofia Blas

2026-04-01

R Markdown

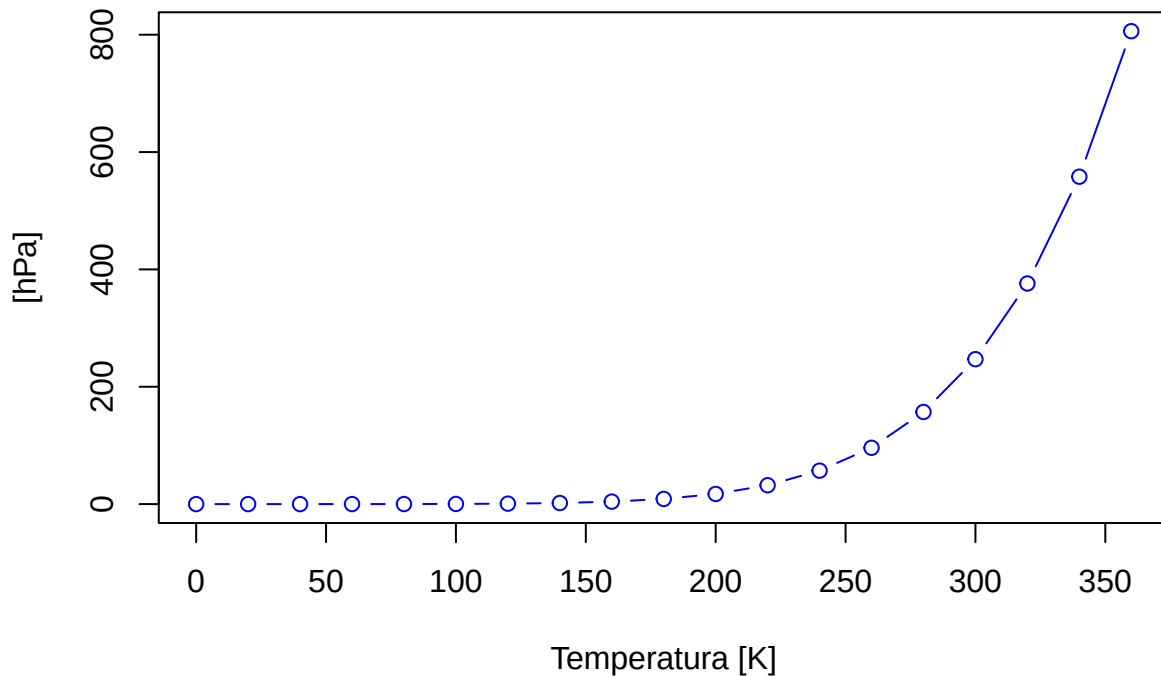
This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>. When you click the *Knit* button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja generalmente con variables que pueden ser matrices. Para escribir un código oprimimos la tecla +C en verde y R.

```
plot(pressure,main="Presión del gas ideal",ylab="Presión  
[hPa]",xlab="Temperatura [K]",type="b", col="blue")
```

Presión del gas ideal



```
a=38  
a=40  
b=60  
c=b-a
```

```
c
## [1] 20
Y=40
alpha<-30
```

Uso de data sets o base de datos internos de R

Usando el comando `data` en la consola obtenemos un listado de distintos datos ya cargados en la base de datos de R. Estos datos están dados en tablas/matrices de las cuales podemos elegir columnas específicas si escribimos signo pesos después del comando de información específico. Por ejemplo `cars$speed`. Nos saltan todas la velocidades que antes la encontrabamos en forma de columna en la matriz de `cars`.

Conclusion

El uso de Posit Cloud representa una solución práctica y accesible para iniciarse en el análisis de datos con R, especialmente en entornos académicos y colaborativos. Su facilidad de acceso, sin necesidad de instalaciones complejas, favorece el aprendizaje y el trabajo en equipo desde cualquier lugar. No obstante, al depender de internet y contar con recursos limitados, puede no ser la opción más adecuada para proyectos de gran escala o que requieran alto rendimiento computacional. En consecuencia, se posiciona como una herramienta ideal para la formación, la experimentación y el desarrollo de proyectos intermedios, mientras que los trabajos más exigentes pueden beneficiarse de un entorno local más robusto.

Tabla comparativa: Posit Cloud vs R local

Característica	Posit Cloud	R local (instalado en PC)
Instalación	No requiere instalación	Requiere instalación de R y RStudio
Acceso	Desde cualquier dispositivo con internet	Solo desde la computadora instalada
Conexión a internet	Obligatoria	No obligatoria
Recursos computacionales	Limitados (según el plan)	Dependen del hardware de la PC
Rendimiento	Adecuado para proyectos pequeños/medianos	Mejor para proyectos grandes o pesados
Trabajo colaborativo	Muy sencillo (compartir proyectos online)	Requiere herramientas externas (Git)
Actualizaciones	Automáticas	Manuales
Ideal para	Aprendizaje y prototipado	Desarrollo profesional y análisis intensivo

Data set en R

¿Qué es un dataset? Es una colección organizada de información, generalmente estructurada en forma de tabla para poder analizarla fácilmente.

Tipos de datasets en R:

Internos → vienen incluidos en R o paquetes Ej: `iris`, `mtcars` **Externos** → archivos que importás Ej: CSV, Excel, TXT

Datasets internos -Ya están disponibles sin descargar -Se usan para aprender y practicar

Comandos básicos:

```
data()      # lista todos los datasets
data(iris)  # carga uno
```

Datasets externos -Se cargan desde archivos

Ejemplos:

```
read.csv("archivo.csv")
read.table("archivo.txt")
```

```
head(datos)      # primeras filas
summary(datos)   # resumen estadístico
str(datos)       # estructura
```

Exploración de datos

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que se le asigne. Se coloca plot y entre paréntesis lo que se quiera graficar. Sepando con comas se van agregando los cambios que se le quieran hacer al gráfico, como el título, lo que esta escrito sobre los ejes, etc. Si no sabemos como es el comando para cambiar algun parámetro del gráfico lo podemos buscar escribiendo en la consola signo de pregunta? seguido del comando plot y explicara en detalle el comando y sus variantes. En la consola podemos escribir y haciendo enter visualizar en la pestaña de la derecha como se vería. Si quedó acorde a lo que queríamos lo copiamos y pegamos en el pestaña de la izquierda superior Source, aquí es donde se hacen todos los cambios que se verán reflejados en el documento que estamos haciendo. Lo que se haga en la consola no queda plasmado.

Descarga de documentos

Tenemos distintas opciones para tejer el documento, para usarlas hay que oprimir el triangulo del ovillo de lana *Knit* y veremos como se despliegan distintas opciones: Knit to HTML nos dará una pagina web, Knit to PDF nos permitirá descargar un PDF, Knit to WORD nos permitirá descargar un archivo WORD.

Funciones estadísticas

Para utilizar funciones estadísticas debemos colocar un código, por lo tanto, debemos oprimir +C y luego R, se nos despliega el espacio para escribir el código. Allí escribimos el comando rnorm

```
z1<-rnorm(351,22,5)
z1
```

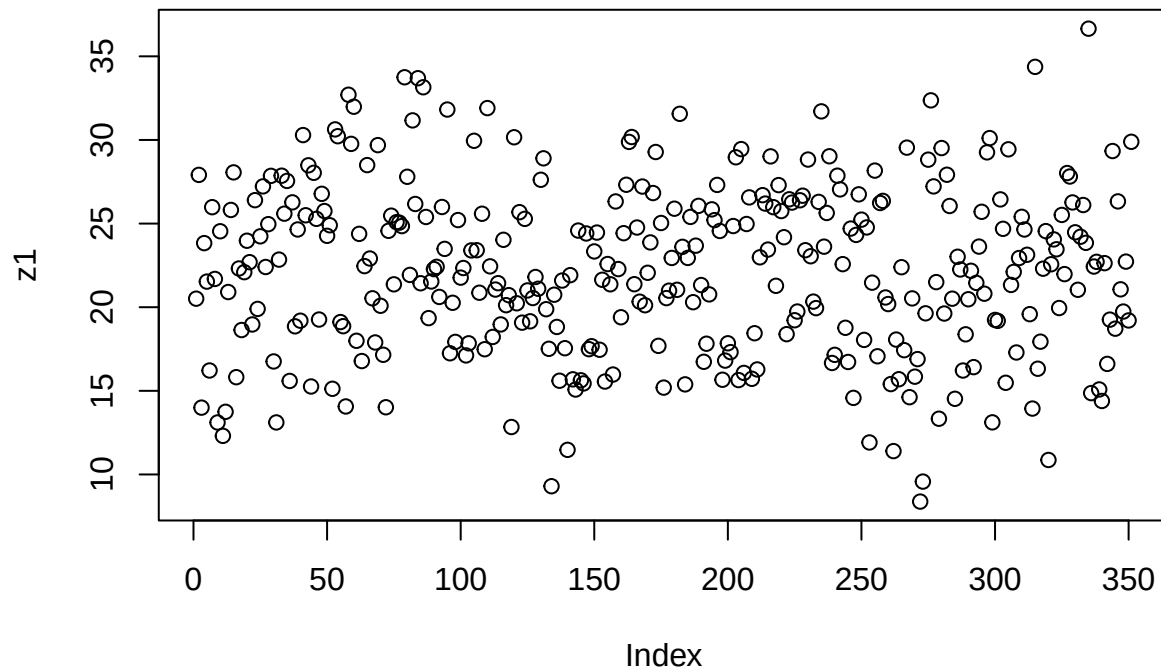
```
## [1] 20.510764 27.911712 13.995127 23.832149 21.531205 16.217228 25.981086
## [8] 21.689929 13.108927 24.541275 12.307928 13.746038 20.906791 25.815553
## [15] 28.061125 15.816484 22.315601 18.635606 22.094765 23.975738 22.693809
## [22] 18.968185 26.400210 19.898375 24.238792 27.224087 22.404438 24.965204
## [29] 27.854913 16.757073 13.110644 22.846641 27.864407 25.593995 27.551786
## [36] 15.591735 26.269816 18.857571 24.648193 19.192887 30.292125 25.498064
## [43] 28.484452 15.259115 28.036372 25.281669 19.262739 26.779687 25.750323
## [50] 24.268120 24.900673 15.122642 30.632406 30.223711 19.112560 18.864430
## [57] 14.065162 32.705150 29.765899 31.987249 17.992278 24.385486 16.781292
## [64] 22.457898 28.497427 22.910962 20.531997 17.892933 29.691397 20.086720
## [71] 17.154905 14.017734 24.567838 25.473198 21.368060 25.084967 25.058078
## [78] 24.850702 33.749523 27.798130 21.931173 31.171400 26.172875 33.692116
## [85] 21.431159 33.150766 25.396035 19.347487 21.538320 22.278811 22.414496
## [92] 20.614293 25.992657 23.486870 31.818896 17.245817 20.265109 17.926527
```

```

## [99] 25.209973 21.768020 22.343128 17.116930 17.841117 23.396650 29.951372
## [106] 23.408043 20.866025 25.587791 17.487910 31.905145 22.442560 18.222053
## [113] 21.055541 21.436720 18.970064 24.031058 20.121340 20.720897 12.836143
## [120] 30.160697 20.234453 25.682643 19.077040 25.282126 21.002740 19.156883
## [127] 20.549386 21.806579 21.103558 27.619235 28.902094 19.880943 17.512819
## [134] 9.296260 20.747197 18.820049 15.604395 21.593906 17.549400 11.471734
## [141] 21.916521 15.687058 15.078659 24.582044 15.630548 15.443275 24.409392
## [148] 17.492877 17.668122 23.336413 24.463326 17.457404 21.640533 15.554199
## [155] 22.582211 21.371040 15.973529 26.323446 22.280024 19.399323 24.426909
## [162] 27.321481 29.884493 30.175458 21.376376 24.764732 20.330304 27.214884
## [169] 20.117695 22.058989 23.872850 26.825476 29.279297 17.689047 25.034741
## [176] 15.193609 20.544041 20.988893 22.944703 25.890493 21.041239 31.562967
## [183] 23.610057 15.386035 22.945839 25.399301 20.299805 23.680656 26.058163
## [190] 21.324294 16.736504 17.818044 20.767185 25.851982 25.205644 27.312052
## [197] 24.559049 15.663729 16.808116 17.842708 17.312870 24.853171 28.963500
## [204] 15.644151 29.453096 16.071817 24.974128 26.565285 15.718910 18.444671
## [211] 16.278965 22.983808 26.699343 26.195987 23.445445 29.018673 25.982916
## [218] 21.279049 27.300877 25.749763 24.185072 18.387402 26.452442 26.257096
## [225] 19.220505 19.742599 26.393352 26.667236 23.408032 28.828167 23.047350
## [232] 20.334430 19.945808 26.296135 31.709468 23.620586 25.634638 29.025046
## [239] 16.664974 17.151962 27.863435 27.043505 22.575579 18.769712 16.725526
## [246] 24.702623 14.579788 24.321950 26.748358 25.235432 18.044784 24.770068
## [253] 11.913737 21.468443 28.168898 17.065661 26.230258 26.356502 20.581489
## [260] 20.188360 15.404086 11.396144 18.064080 15.693013 22.391514 17.431046
## [267] 29.541566 14.620683 20.527037 15.846500 16.897498 8.378906 9.574312
## [274] 19.634831 28.821571 32.368411 27.222777 21.500825 13.332265 29.515711
## [281] 19.622288 27.921591 26.052891 20.509681 14.524008 23.016816 22.256961
## [288] 16.214869 18.379184 20.475316 22.178697 16.417066 21.455208 23.624882
## [295] 25.705308 20.808874 29.258610 30.111855 13.115319 19.240003 19.184119
## [302] 26.446712 24.687389 15.481171 29.441405 21.331686 22.111611 17.293530
## [309] 22.940304 25.403976 24.646490 23.135752 19.584765 13.939522 34.370142
## [316] 16.321896 17.940875 22.304735 24.550844 10.866376 22.568333 24.054690
## [323] 23.462534 19.951320 25.516143 21.978366 28.021710 27.814684 26.265041
## [330] 24.478869 21.041038 24.214510 26.110505 23.848476 36.653323 14.864575
## [337] 22.430997 22.713441 15.082759 14.394396 22.634243 16.609612 19.256863
## [344] 29.338142 18.716164 26.327194 21.068044 19.739768 22.725967 19.209252
## [351] 29.889789

```

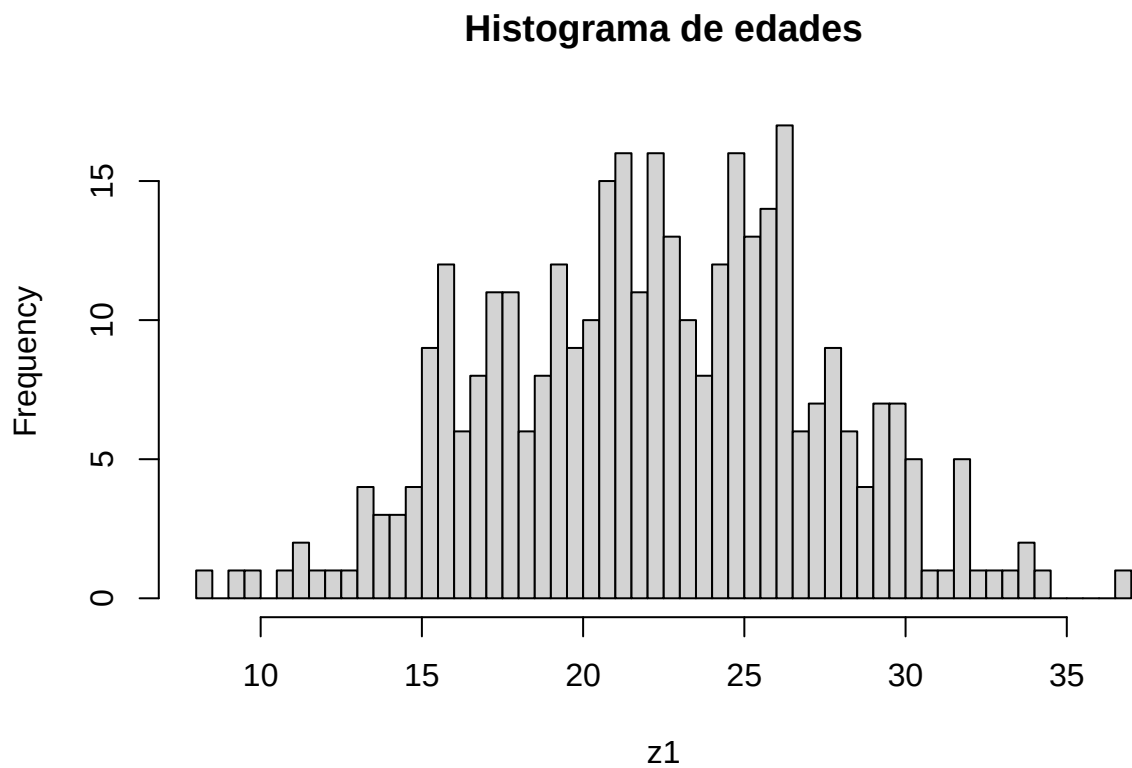
```
plot(z1)
```



hist es

un comando para hacer un histograma.

```
hist(z1, main="Histograma de edades", breaks=50)
```



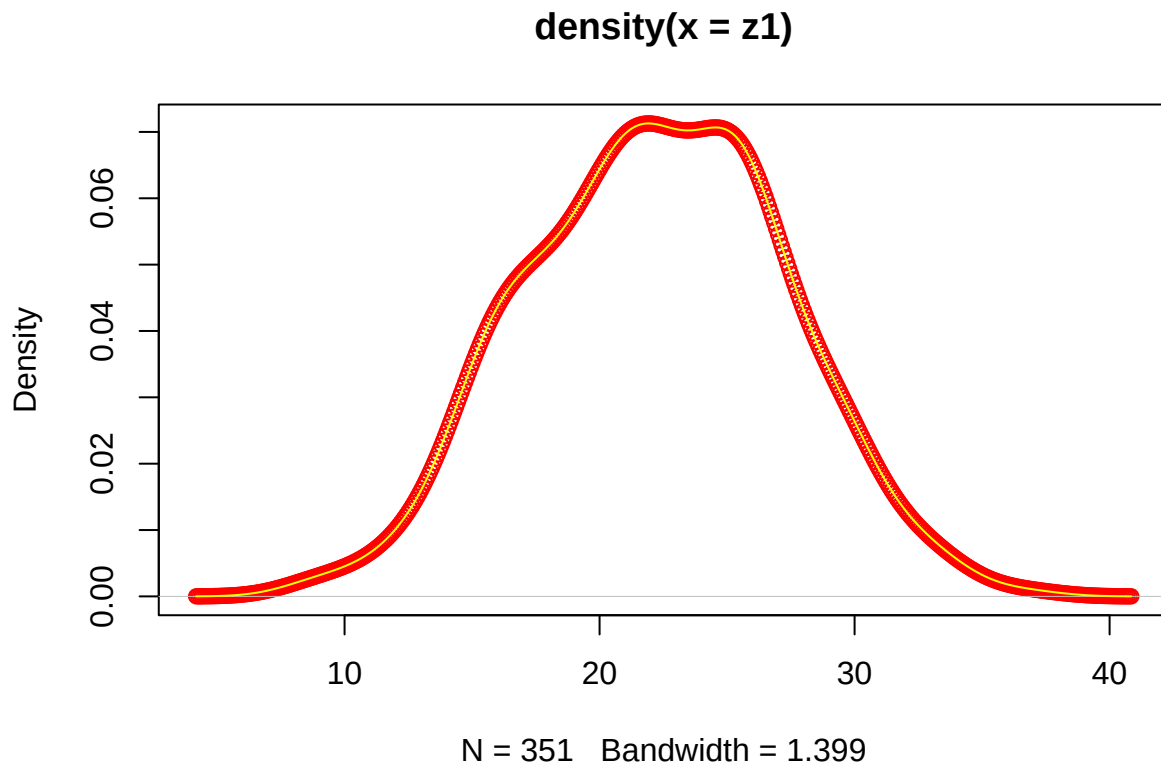
density es un comando para hacer un grafico similar a una campana de Gauss pero no es perfecto.

```
density(z1)
```

```
##  
## Call:
```

```
## density.default(x = z1)
##
## Data: z1 (351 obs.); Bandwidth 'bw' = 1.399
##
##      x              y
## Min.   : 4.182    Min.   :9.118e-06
## 1st Qu.:13.349    1st Qu.:2.146e-03
## Median :22.516    Median :1.622e-02
## Mean   :22.516    Mean    :2.722e-02
## 3rd Qu.:31.683    3rd Qu.:5.272e-02
## Max.   :40.850    Max.    :7.128e-02
```

```
plot(density(z1),type="p",col="red")
lines(density(z1),col="yellow")
```



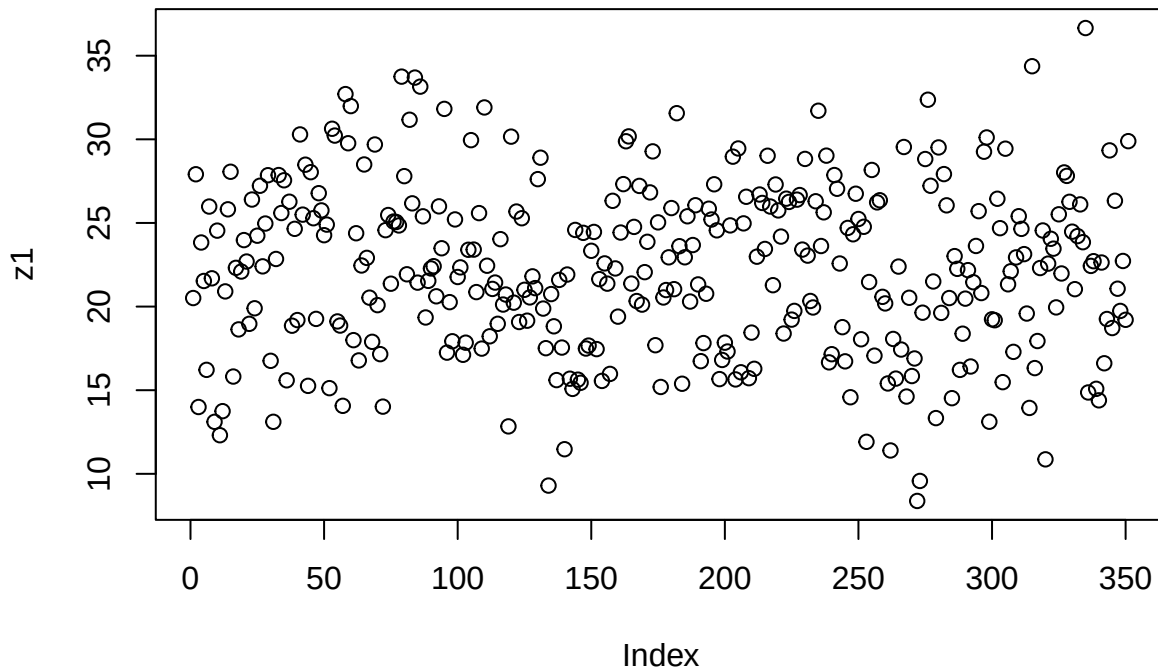
Normal {stats} R Documentation

The Normal Distribution

Description Density, distribution function, quantile function and random generation for the normal distribution with mean equal to mean and standard deviation equal to sd.

Usage `dnorm(x, mean = 0, sd = 1, log = FALSE)` `pnorm(q, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)` `qnorm(p, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)` `rnorm(n, mean = 0, sd = 1)`
Arguments `x`, `q` vector of quantiles. `p` vector of probabilities. `n` number of observations. If `length(n) > 1`, the length is taken to be the number required.

```
plot(z1)
```



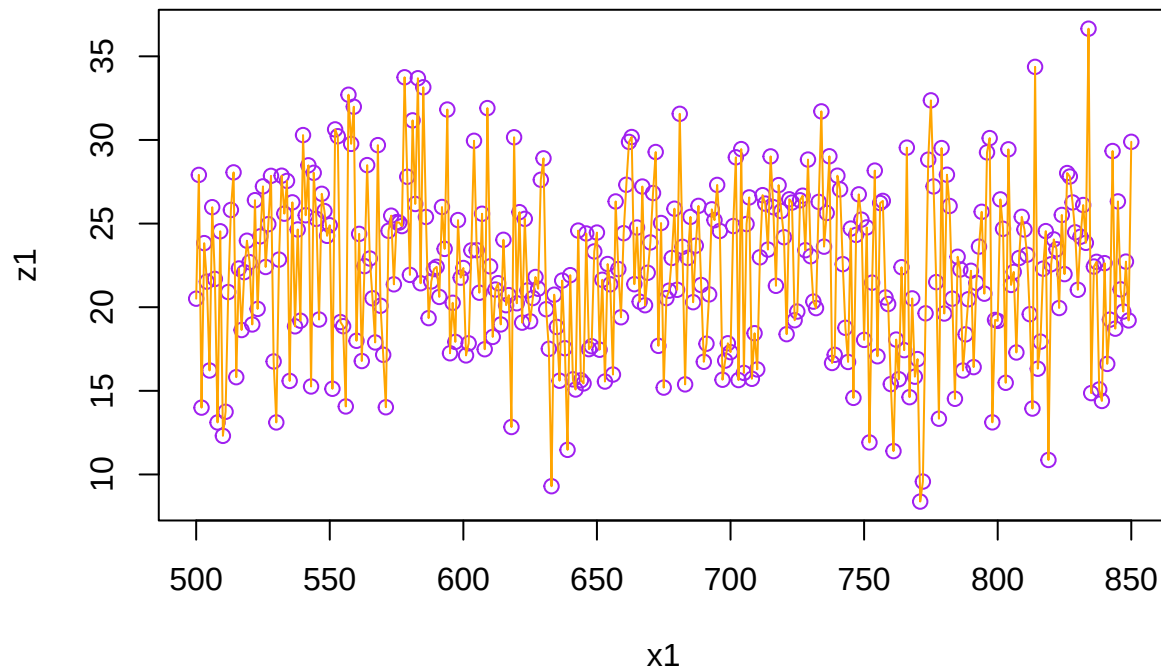
```
w1<-length(z1)
w1
```

```
## [1] 351
```

```
x1<-(500:850)
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849 850
```

```
plot(x1, z1, col="purple")
lines(x1, z1, type="l", col="orange")
```



Clase 8 de Abril

Ejercicio 1 de algoritmos

Consigna 1: Las últimas 3 cifras de mi DNI son 035. Crear una variable que tenga ese número.

```
DNI=035
```

Consigna 2: Crear un vector que tenga todos los números desde el 1 hasta el 035.

```
for(i in 1:035)
  print(1:i)
```

```
## [1] 1
## [1] 1 2
## [1] 1 2 3
## [1] 1 2 3 4
## [1] 1 2 3 4 5
## [1] 1 2 3 4 5 6
## [1] 1 2 3 4 5 6 7
## [1] 1 2 3 4 5 6 7 8
## [1] 1 2 3 4 5 6 7 8 9
## [1] 1 2 3 4 5 6 7 8 9 10
## [1] 1 2 3 4 5 6 7 8 9 10 11
## [1] 1 2 3 4 5 6 7 8 9 10 11 12
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```



```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35
```

```
secuencia_DNI <- c()
for(i in 1:035){
  secuencia_DNI <- c(secuencia_DNI, i)
}
```

Ejercicio 2

Consigna 3: Calcular la suma de todos los valores del vector `secuencia_DNI`

```
Total<-0
Valor_final<-length(secuencia_DNI)
for(i in 1:Valor_final){
  Total<-Total+i
}
Total
```

```
## [1] 630
```

```
print(Total)
```

```
## [1] 630
```

Ejercicio 3

Consigna 4: Repetir el ejercicio anterior pero en Python

```
{python} secuenciaDNI = range(1, 036) # incluye 035 total = sum(secuenciaDNI) print(total)
```

Ejercicio 4

Consigna 5: ¿Cuánto tarda en correr el *Ejercicio 3*?

```
system.time(for(i in 1:100) mad(runif(1000)))
```

```
##      user  system elapsed  
## 0.012   0.000   0.012
```

El comando system.time sirve para conocer cuantos milisegundo tarda en ejecutarse un bloque de código

Ejercicio 5

Determinar CPU time used

```
require(stats)  
system.time(for(i in 1:100) mad(runif(1000)))
```

```
##      user  system elapsed  
## 0.011   0.000   0.013
```

```
require(stats)  
DNI= 35  
secuencia= 1:DNI  
total <- 0  
Valor_final <- length(secuencia)  
for (i in 1:Valor_final){  
  total <- total + secuencia[i]  
}  
print(total)
```

```
## [1] 630
```

```
sleep_for_a_minute <- function() { Sys.sleep(14) }  
start_time <- Sys.time()  
sleep_for_a_minute()  
end_time <- Sys.time()  
end_time - start_time
```

```
## Time difference of 14.01492 secs
```

Ejercicio 6: Comparación de performance con system.time()

```
system.time({  
  A <- numeric(50000)  
  for (i in 1:50000) {  
    A[i] <- i * 2  
  }  
})
```

```
##      user  system elapsed  
## 0.004   0.000   0.004
```

```
system.time({  
  B <- seq(from = 2, to = 100000, by = 2)  
})
```

```
##      user  system elapsed  
## 0.001   0.000   0.001
```

Biblioteca tictoc

El uso de una biblioteca en R consiste en incorporar un conjunto de funciones adicionales que amplían las capacidades del lenguaje. Estas bibliotecas contienen procedimientos previamente desarrollados que permiten realizar tareas específicas de manera más sencilla y eficiente. Para poder utilizarlas, primero es necesario instalarlas mediante el comando `install.packages()`, lo que descarga el paquete al entorno de trabajo. Luego, se debe cargar con `library()`, paso que habilita sus funciones para ser utilizadas en la sesión actual.

En el caso de la biblioteca `tictoc`, su finalidad es medir el tiempo de ejecución de un bloque de código. Esta incorpora las funciones `tic()` y `toc()`, que actúan como un cronómetro: la primera inicia la medición y la segunda la finaliza mostrando el tiempo transcurrido. Estas funciones son similares a las utilizadas en Octave o Matlab, manteniendo una lógica de uso sencilla y familiar. Si bien R dispone de herramientas propias para medir tiempos, como `system.time()`, la biblioteca `tictoc` ofrece una alternativa más cómoda y clara, especialmente cuando se desea evaluar y comparar el rendimiento de distintos procedimientos. De este modo, facilita el análisis de eficiencia del código y mejora la organización del trabajo.

```
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
## [1] "...suena el despertador"
toc()
```

Biblioteca rbenchmark

La función `benchmark` del paquete `rbenchmark` se describe en su documentación como un contenedor sencillo alrededor de la función base `system.time()` de R. Esto significa que internamente utiliza el mismo mecanismo para medir el tiempo de ejecución, pero lo presenta de una manera más práctica y organizada para el usuario.

Aunque `system.time()` permite medir cuánto tarda una expresión en ejecutarse, su uso repetido puede volverse poco cómodo cuando se desean comparar varias expresiones o realizar múltiples repeticiones de cada una. En cambio, `benchmark` simplifica este proceso, ya que permite evaluar varias expresiones dentro de una sola llamada, indicando además cuántas repeticiones se desean realizar. Otra ventaja importante es que los resultados obtenidos se devuelven en forma de data frame, lo que facilita su lectura, comparación y posterior análisis. De este modo, la función no solo automatiza la medición de tiempos, sino que también organiza la información de manera clara y estructurada, aportando mayor comodidad y eficiencia al trabajo de evaluación de rendimiento en R.

Implementación de una serie Fibonacci o Fibonacci

En matemáticas, la sucesión de Fibonacci es una sucesión infinita de números naturales que comienza de la siguiente manera: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ... La característica principal de esta sucesión es que, a partir de los dos primeros términos (0 y 1), cada número se obtiene sumando los dos anteriores. Los elementos que forman esta sucesión se denominan números de Fibonacci. Esta sucesión fue introducida en Europa por Leonardo de Pisa, matemático italiano del siglo XIII, más conocido como Fibonacci, en su obra *Liber Abaci*. Aunque ya era conocida en matemáticas orientales, su difusión en Europa se debe a este autor. La sucesión de Fibonacci posee numerosas aplicaciones en diversas áreas, como la matemática, la informática, la teoría de juegos e incluso en fenómenos naturales y modelos de crecimiento. Su importancia radica tanto en su sencillez como en las múltiples propiedades y relaciones que presenta dentro del estudio de los números.

Consigna 8: ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```

# Inicialización
a <- 0
b <- 1
serie <- c(a, b) # guardamos la serie
contador <- 2
# ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
  siguiente <- a + b
  a <- b
  b <- siguiente
  serie <- c(serie, b) # agregamos a la serie
  contador <- contador + 1
}
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")

```

```
## Cantidad de iteraciones: 9
```

```
cat("Primer número mayor a 20:", b, "\n")
```

```
## Primer número mayor a 20: 21
```

```
cat("Serie de Fibonacci hasta ese número:\n")
```

```
## Serie de Fibonacci hasta ese número:
```

```
print(serie)
```

```
## [1] 0 1 1 2 3 5 8 13 21
```

Consigna 9: Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```

# Instalar paquete
if (!require(microbenchmark)) {
  install.packages("microbenchmark")
  library(microbenchmark)
}

```

```
## Loading required package: microbenchmark
```

```

# Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
#Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
}

```

```

return(x)
}
#Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
  burbuja = bubble_sort(vector),
  sort_R = sort(vector),
  times = 5
)
print(resultado)

## Unit: microseconds
##      expr      min       lq      mean     median      uq
## burbuja 20853791.09 20940779.79 2.105178e+07 21027522.060 21124935.828
##  sort_R    571.51    617.57 7.233566e+02    719.521    739.361
##           max neval
## 21311872.489    5
##    968.821    5

```

La penitencia de Newton

Algunos historiadores relatan que Isaac Newton, durante su etapa escolar, tuvo un maestro sumamente estricto que exigía absoluta atención en clase. Ante cualquier distracción o falta de disciplina, imponía como castigo una tarea particularmente tediosa: calcular para el día siguiente la suma de todos los números desde 1 hasta un número elegido al azar. A simple vista, esta consigna implicaba realizar largas sumas parciales, una por una, lo que podía demandar horas de trabajo. El profesor, precavido, ya tenía resueltos estos cálculos y los llevaba anotados en un cuaderno voluminoso, similar en utilidad a una tabla de logaritmos, que transportaba cuidadosamente en su portafolio. La anécdota cobra relevancia porque, según la tradición, el joven Newton habría encontrado una forma más rápida y elegante de realizar esas sumas, descubriendo un patrón que permitía obtener el resultado sin necesidad de sumar término por término. Más allá de su veracidad histórica, el relato ilustra cómo la observación y el razonamiento pueden transformar una tarea repetitiva en un problema matemático con una solución general.

Consigna 10: Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×10^{10} y verifica cuál de los dos es más eficiente.

```

#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
  suma1 <- suma1 + i
}
suma1

## [1] 510555

#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010
suma2 <- n * (n + 1) / 2
suma2

## [1] 510555

#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
# Método 1
system.time({

```

```

suma1 <- 0
for (i in 1:1010){
  suma1 <- suma1 + i
}
})

##      user  system elapsed
##    0.001   0.000   0.002

# Método 2
system.time({
  suma2 <- 1010 * (1010 + 1) / 2
})

```

```

##      user  system elapsed
##         0         0         0

```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1×1010 . El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente.

##Matriz Inversa

```

##Definir la matriz
A <- matrix(c(4, 7,
              2, 6),
            nrow = 2,
            byrow = TRUE)
##Calcular la inversa
A_inv <- solve(A)
##Mostrar resultado
A_inv

```

```

##      [,1] [,2]
## [1,]  0.6 -0.7
## [2,] -0.2  0.4

```

Investigar con los gráficos de violin (de la biblioteca micro como se comporta una computadora usando métodos de ordenamiento “burbuja” y compararlo con el método sort nativo de R .

Medir la performance del método kmean de agrupamiento. Realizar una introducción teórica antes de medir con gráfico de violin.

```

library(microbenchmark)
library(ggplot2)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##      filter, lag

## The following objects are masked from 'package:base':

```

```

##
## intersect, setdiff, setequal, union
# Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)
# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 30
)
# Convertir a ms
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

library(microbenchmark)
library(ggplot2)
library(dplyr)

# 1. Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# 2. Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)

# 3. Benchmark (Medición de performance)

```

```

bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 30
)

# 4. Convertir resultados a milisegundos
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

# 5. Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms, fill = expr)) +
  geom_violin(trim = FALSE, alpha = 0.7, color = "black") +
  geom_boxplot(width = 0.1, fill = "white", outlier.shape = 16, outlier.size = 1) + # <--- ESTE "+" FAL
  scale_y_log10() +
  scale_fill_manual(values = c("bubble" = "#FF6666", "sortR" = "#66B2FF")) +
  labs(title = "Comparación de Eficiencia (Escala Logarítmica)",
       subtitle = "La escala log10 permite visualizar la enorme diferencia de velocidad",
       x = "Algoritmo Evaluado",
       y = "Tiempo (ms) en escala log10",
       fill = "Método") +
  theme_light()

```

